# Chapter 1

# Employee Leave Management System (LeaveFlow)

## 1.1 System Architecture

LeaveFlow follows a layered architecture consisting of a web interface, a Flask application, a C++ processing component, and local data storage. Flask manages authentication, role-based access control, and leave records. It invokes the compiled C++ backend for leave validation and processing.

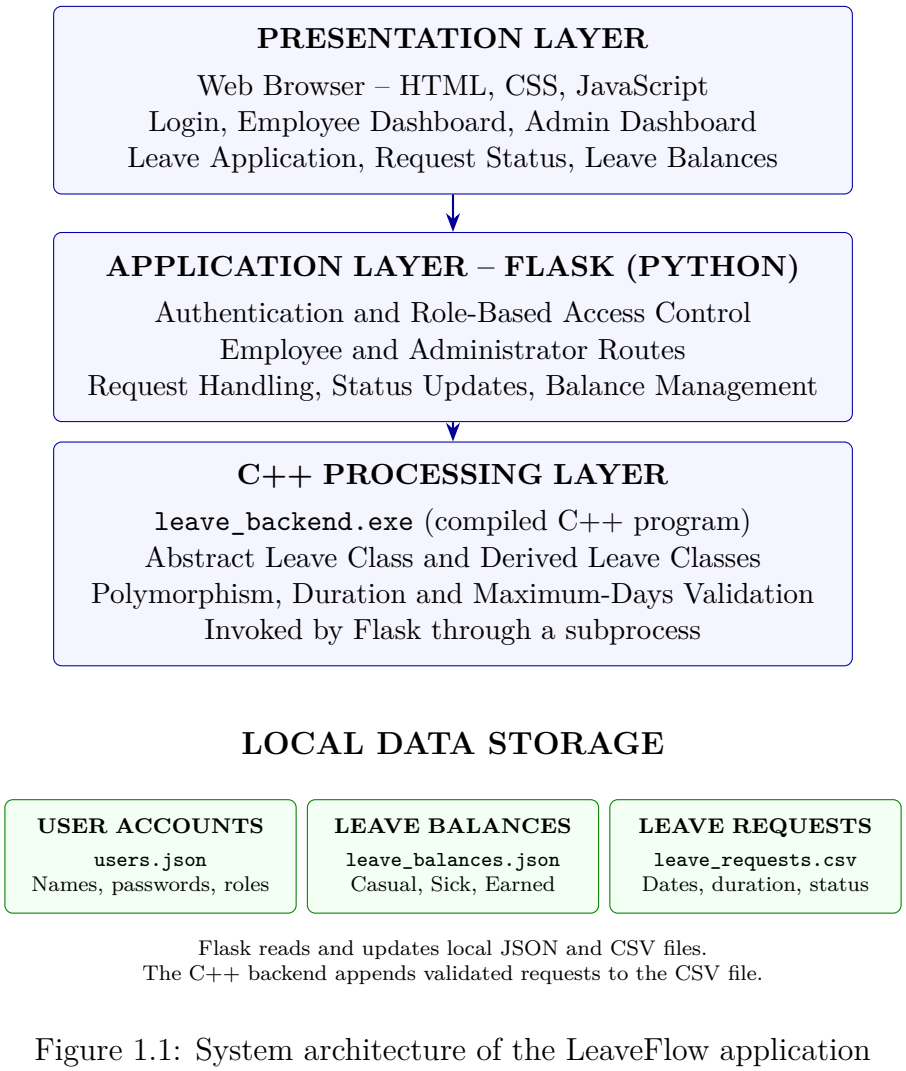



Figure 1.1: System architecture of the LeaveFlow application

## 1.2 Use-Case Diagram

The use-case diagram represents the interactions between the two main actors, Employee and Administrator, and the LeaveFlow system. Employees can apply for leave, view their balances, and track their requests. Administrators can add employees, view and filter requests, and approve or reject pending applications.

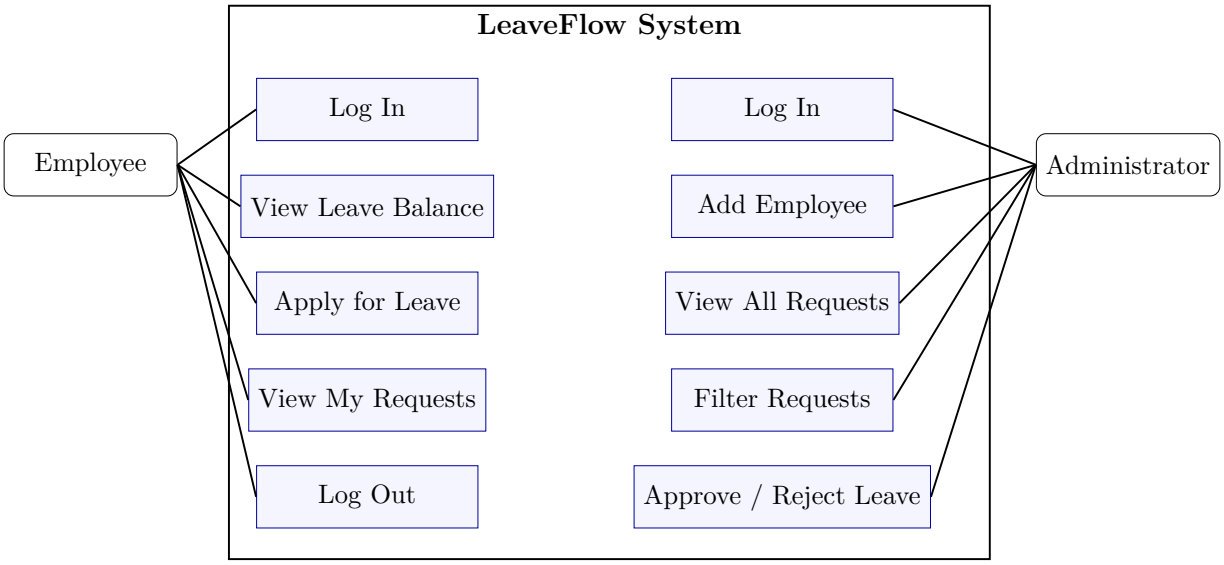



Figure 1.2: Use-case diagram of the LeaveFlow system

## 1.3 Sequence Diagram

The sequence diagram illustrates the flow of a leave application from the employee to the backend components. The employee submits the application through the web interface. Flask validates the request fields and invokes the C++ executable to validate the dates, calculate the duration, and check the maximum permitted leave duration. If validation succeeds, the C++ backend saves the request with a pending status and returns the result to Flask, which communicates the outcome to the browser.

The activity diagram summarizes the administrator's decision process after a leave request is submitted. The administrator reviews a pending request and either rejects it or attempts to approve it. Approval depends on the employee having sufficient leave balance; if the balance is insufficient, the request remains pending. This workflow distinguishes a rejected request from an approved request and shows why a balance check is required before deducting leave.

The class diagram presents the main object-oriented components used in LeaveFlow. The abstract `User` class represents shared user details and operations, while `Employee` and `Admin` provide role-specific behaviour. The `LeaveBalance` class manages the available leave categories. The abstract `Leave` class defines common leave operations, and `CasualLeave`, `SickLeave`, and `EarnedLeave` provide leave-type-specific behaviour, including maximum duration limits.

## 1.4 Implementation Procedure

### 1.4.1 Frontend Design

The frontend of LeaveFlow provides a user-friendly web interface for employees and administrators. It is developed using HTML and CSS, with JavaScript used where required for user

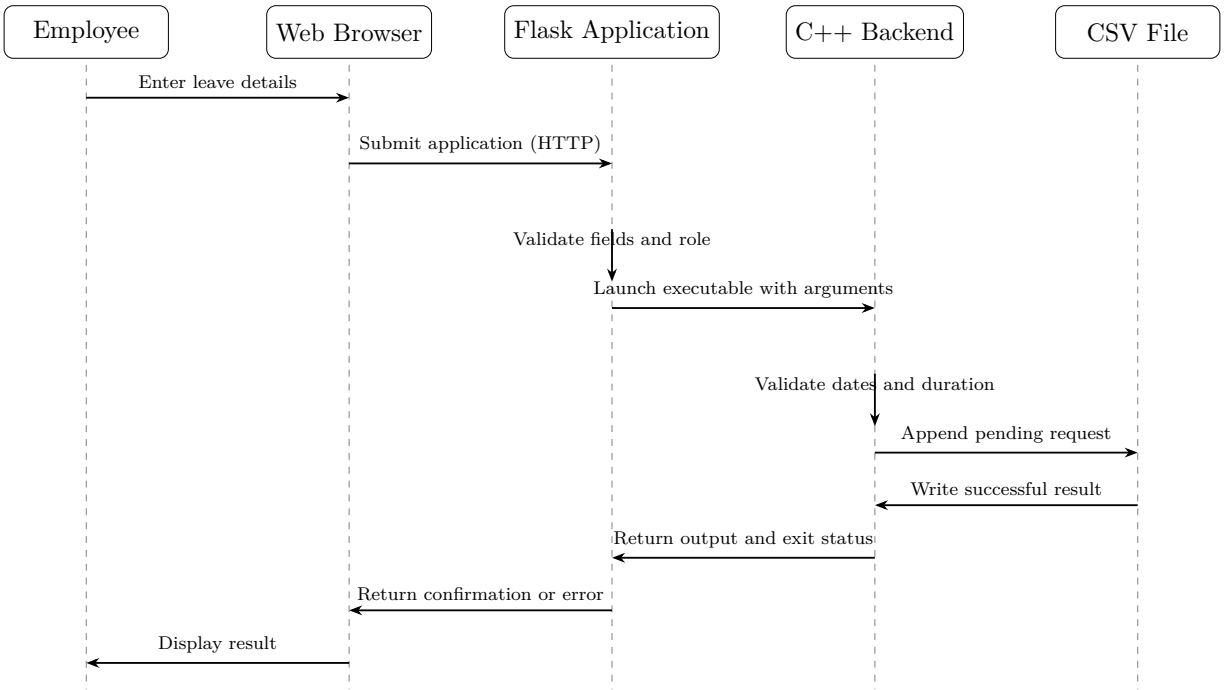



Figure 1.3: Sequence diagram for submitting a leave request

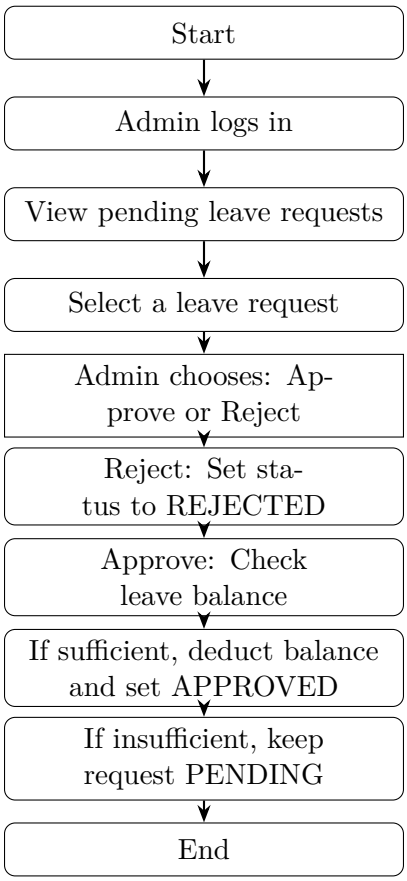



Figure 1.4: Activity diagram for leave request processing

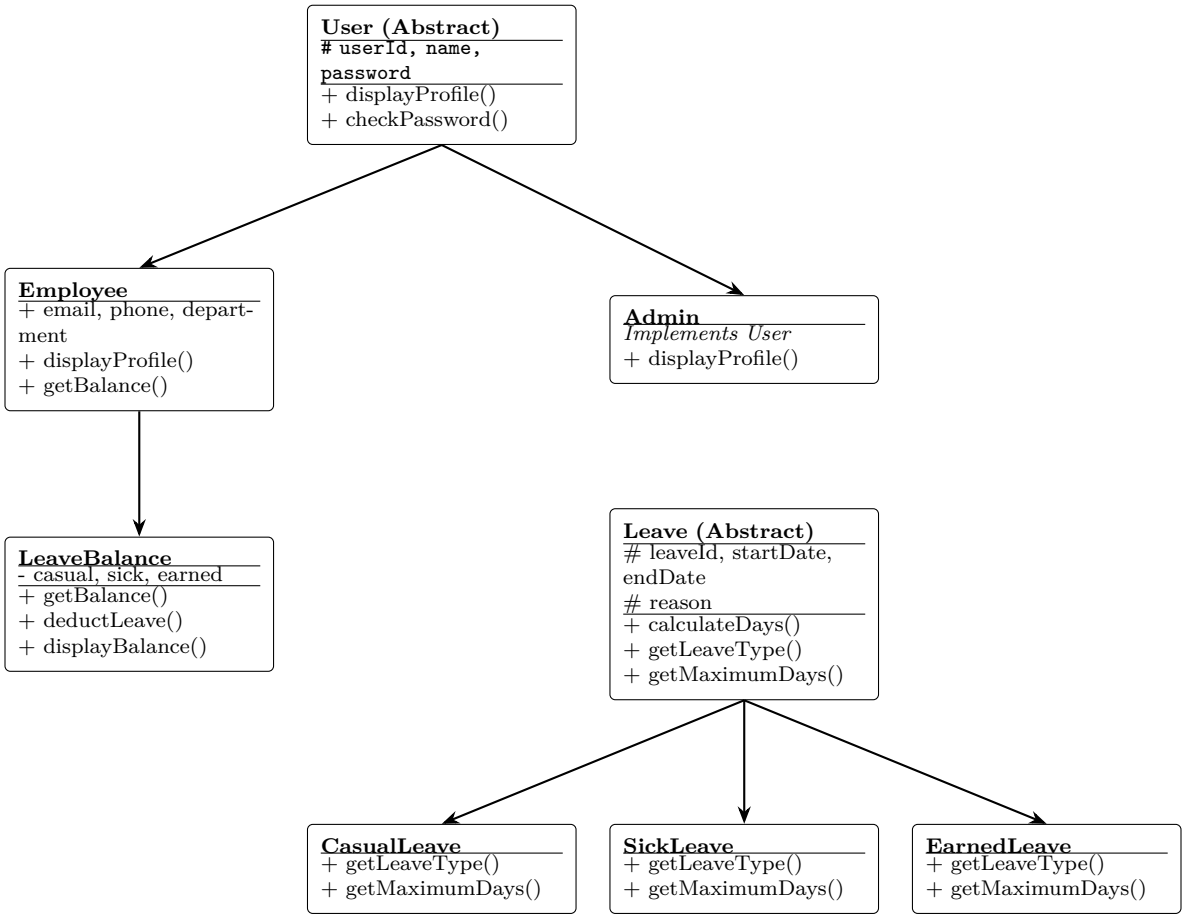



Figure 1.5: Class diagram showing the main C++ classes in LeaveFlow

interactions. The interface includes a login page, an employee dashboard, a leave application form, and an administrative dashboard.

The login page allows users to enter their credentials and select their role. After successful authentication, employees can view their leave balances, submit leave requests, and track their request status. Administrators can access employee management and leave request review features.

The frontend communicates with the Flask application, which processes user requests and connects the interface to the backend logic and data storage.

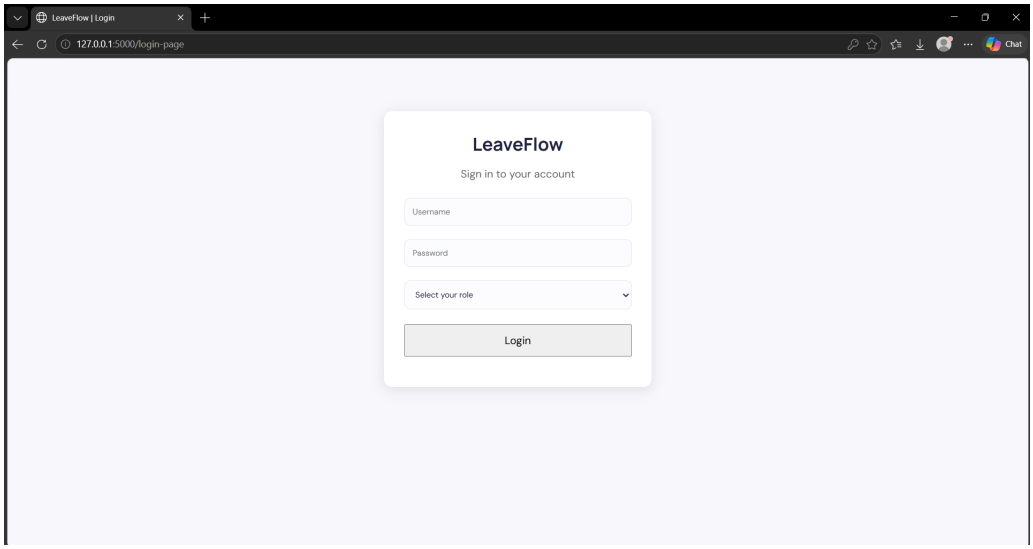



Figure 1.6: LeaveFlow login interface

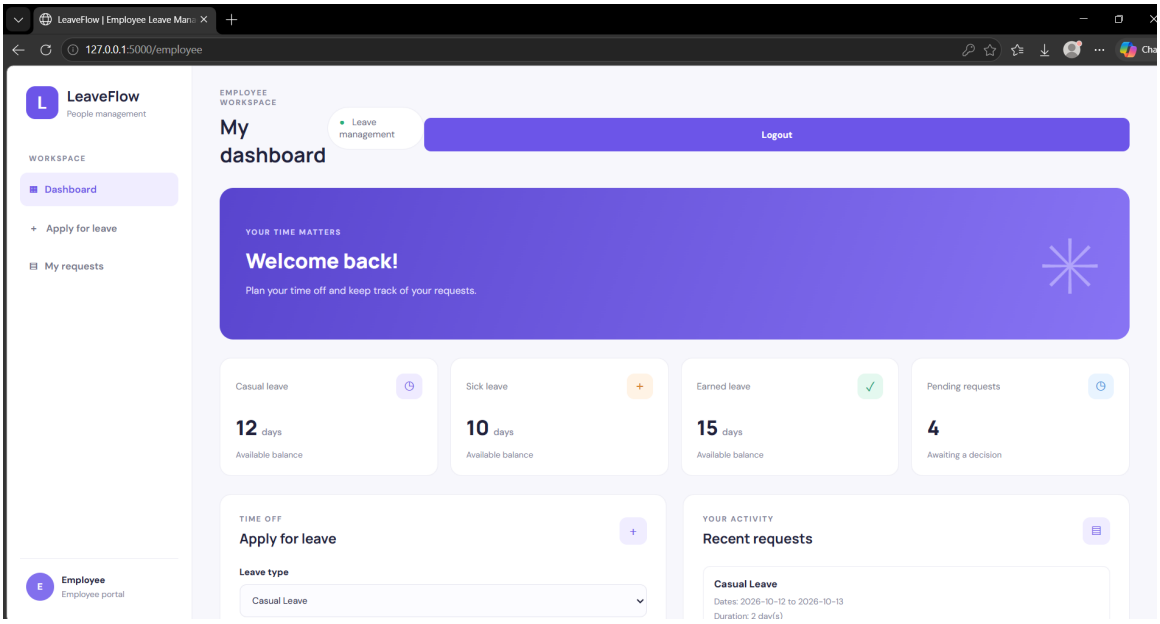



Figure 1.7: Employee dashboard displaying leave balances and request information

### 1.4.2 Login and Role-Based Security

LeaveFlow provides a login interface for employees and administrators. Users enter their credentials and select their role. The Flask application validates the login details using the user

information stored in `users.json`. After successful authentication, a session is established to maintain the user's login state.

Role-based access control restricts access to different features. Employees can submit leave requests, view their own requests, and check their leave balances. Administrators can view all leave requests, manage employee accounts, and approve or reject requests.

The application checks the user's session and role before allowing access to protected routes. This prevents employees from accessing administrator-only operations through restricted URLs.

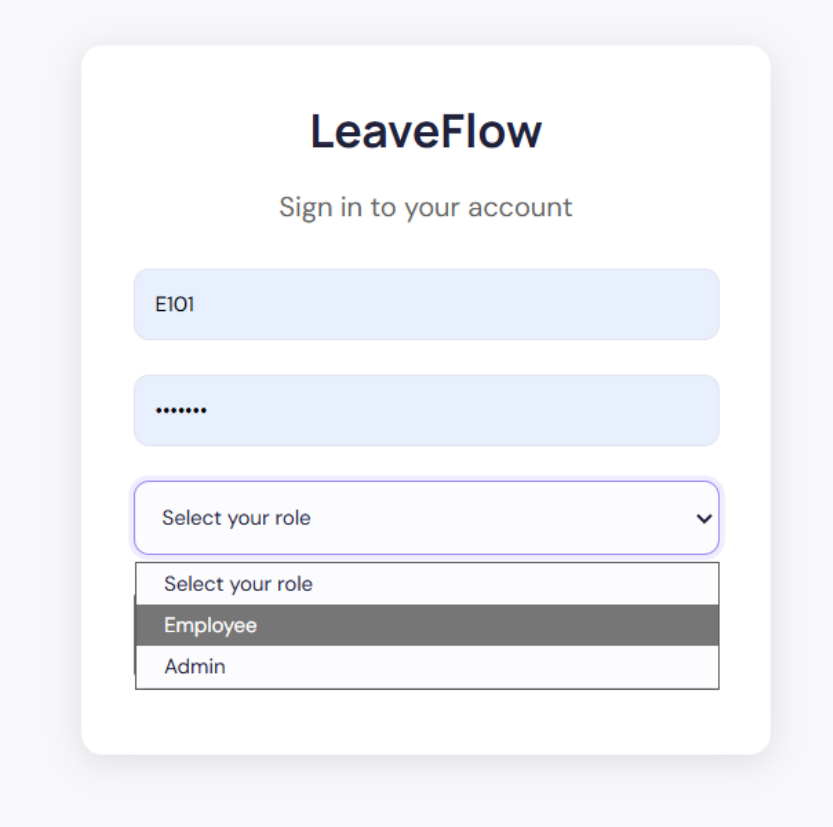



Figure 1.8: Login interface showing credential fields and role selection

The following simplified Python example illustrates session-based access control:

```
from flask import session, redirect, url_for

@app.route('/requests')
def requests_page():
    if 'username' not in session:
        return redirect(url_for('login'))

    if session.get('role') != 'admin':
        return "Access denied", 403

    return render_template('requests.html')
```

The session check ensures that the user is logged in, while the role check restricts the request-management page to administrators.

### 1.4.3 Leave Request Submission

Employees can submit leave requests by selecting the leave type, entering the start and end dates, and providing a reason. The Flask application receives the submitted details and invokes the C++ backend executable using Python's `subprocess` module.

The C++ backend validates the dates, calculates the leave duration, and checks the maximum permitted duration for the selected leave type. If the request is valid, it is stored in `leave_requests.csv` with an initial status of `PENDING`. The result is returned to Flask, which displays a confirmation or error message to the employee.

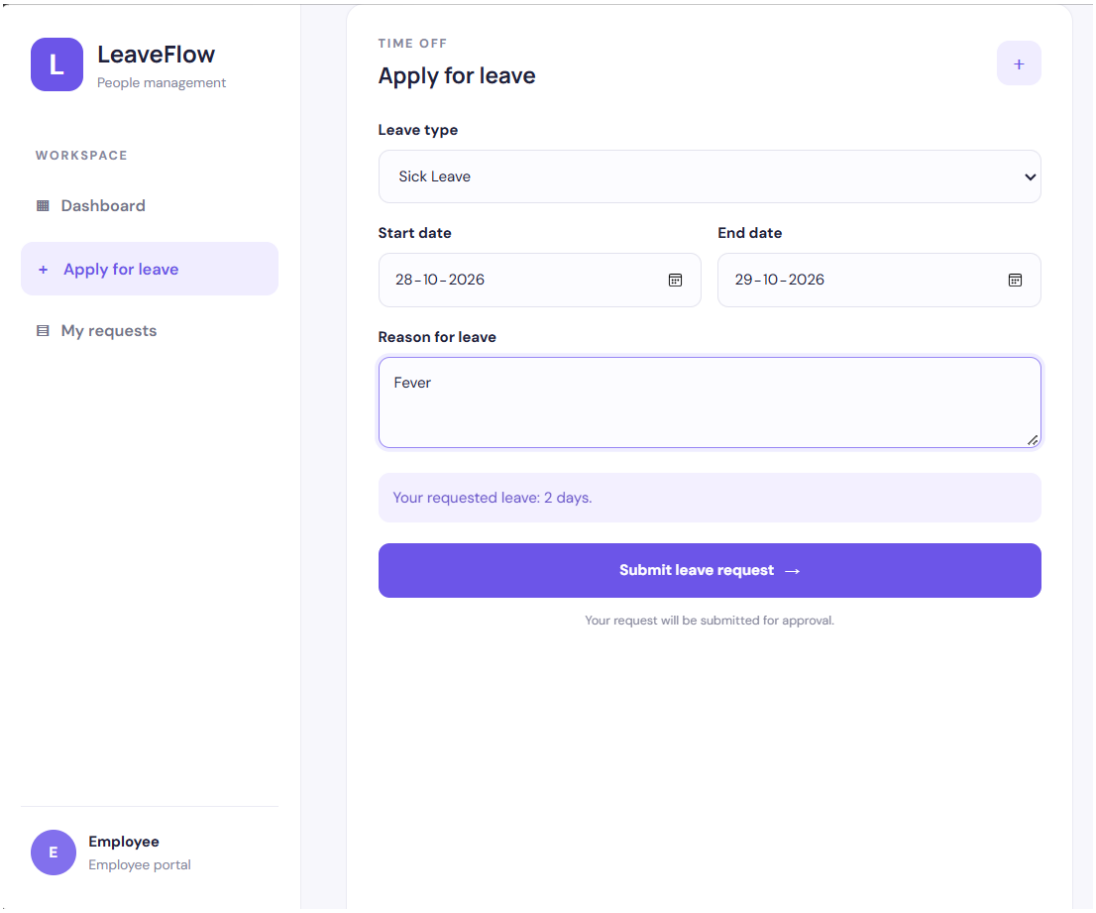



Figure 1.9: Leave application form used by employees

The following simplified Python example illustrates how Flask invokes the C++ backend:

```
import subprocess

result = subprocess.run(
    ["backend/leave_backend.exe",
     employee_id, leave_type,
     start_date, end_date, reason],
    capture_output=True,
    text=True
)
```

The backend's output and exit status are used by the Flask application to determine whether the request was processed successfully. This integration combines the web interface with the C++ leave-validation logic.

### 1.4.4 Admin Approval, Rejection, and Leave-Balance Updates

The administrator reviews pending leave requests through the admin dashboard. Each request displays the employee ID, leave type, dates, duration, reason, and current status. The administrator can approve or reject a request using the available controls.

When a request is approved, the Flask application verifies the employee's available leave balance. If sufficient balance is available, the requested number of days is deducted and the request status is changed to `APPROVED`. If the administrator rejects the request, its status becomes `REJECTED`, and the employee's balance remains unchanged.

The updated request status is displayed in the employee's request history. This allows employees to track the outcome of their applications. Keeping approval and rejection actions in the administrator interface also gives the workflow a clear separation of responsibilities: employees submit requests, while administrators review them. The balance is deducted only when a request is approved and the available balance is sufficient.

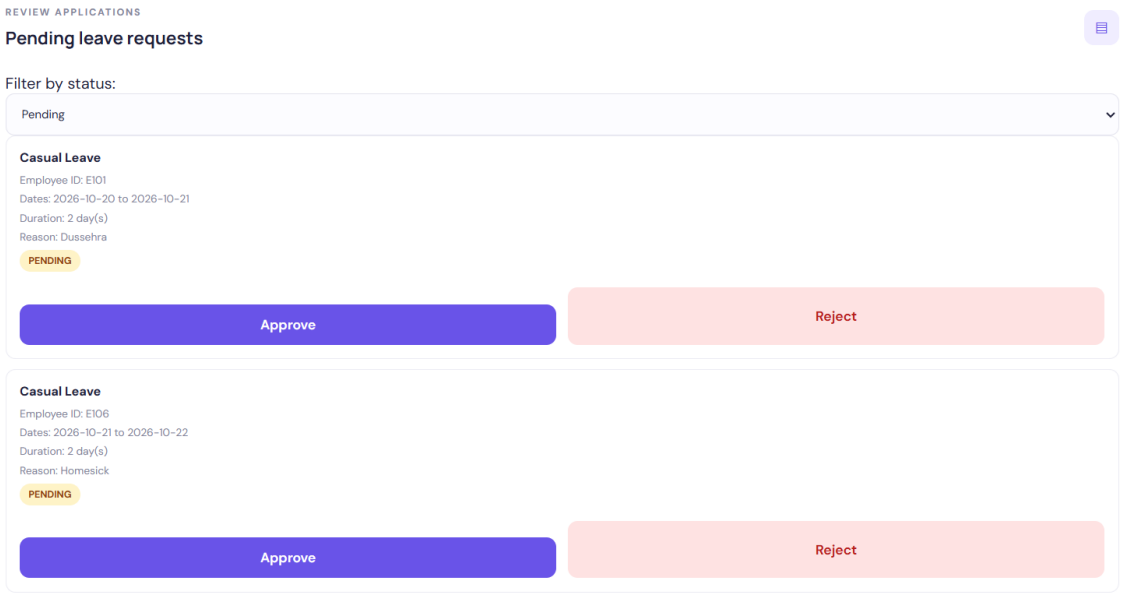



Figure 1.10: Administrator interface for approving or rejecting pending leave requests

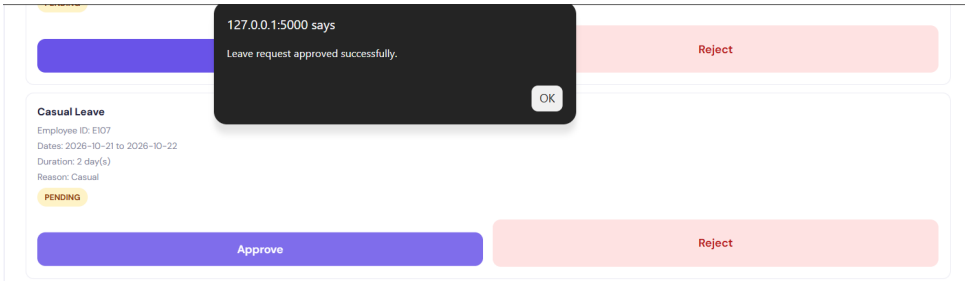



Figure 1.11: Successful leave request approval confirmation

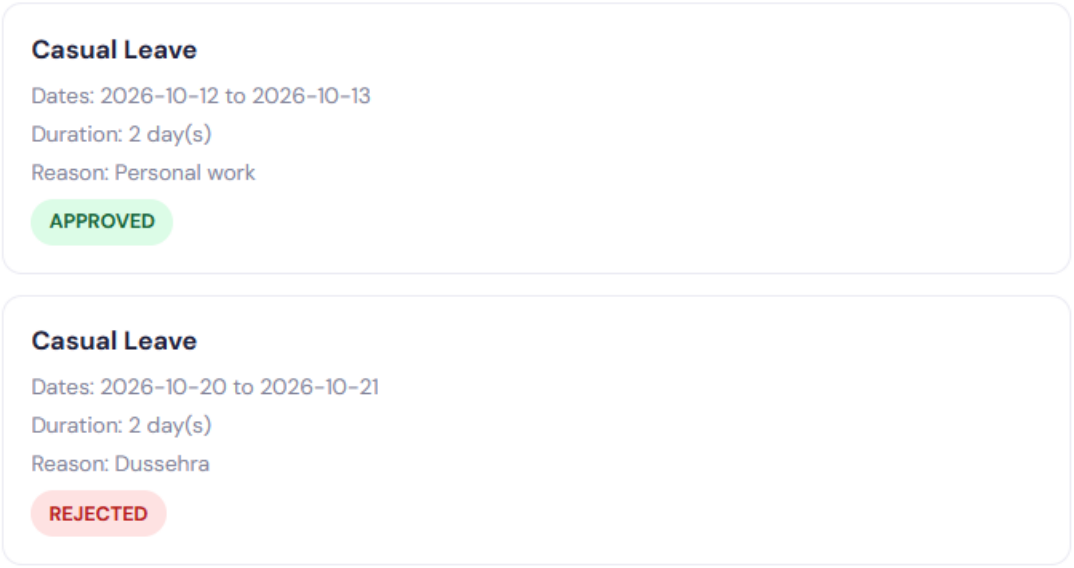



Figure 1.12: Employee request history displaying approved and rejected requests

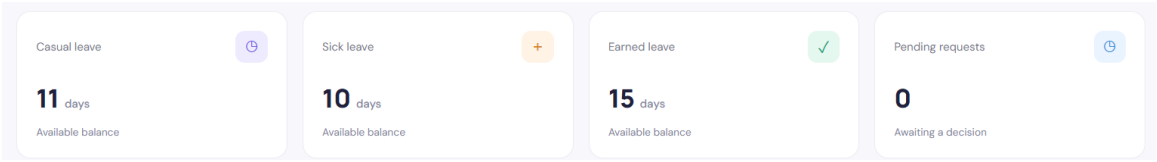



Figure 1.13: Updated leave balances after approval of a leave request

### 1.4.5 Data Storage and Management

LeaveFlow uses JSON and CSV files for persistent local data storage. These files maintain employee information, leave balances, and leave request records.

- **users.json:** Stores employee account information used by the Flask application during authentication.
- **leave_balances.json:** Stores the available casual, sick, and earned leave balances for employees. The application updates these balances when leave requests are approved.
- **leave_requests.csv:** Stores submitted leave requests, including employee ID, leave type, start date, end date, duration, reason, and request status.

The Flask application reads and updates these files to display request histories, manage employee balances, and process administrative decisions. The C++ backend appends valid leave requests to the CSV file with an initial status of `PENDING`.

During approval or rejection, the application updates the relevant records. Temporary files are used before replacing the original files during updates, helping reduce the risk of partially written data.

This file-based approach is suitable for a small academic project because it is simple to implement and does not require a separate database server. Each file has a distinct responsibility: account data supports authentication, balance data supports leave availability checks, and request records preserve the history and current status of applications. Because these files are local, the application is best suited to a small demonstration or academic deployment rather than a high-concurrency production environment.

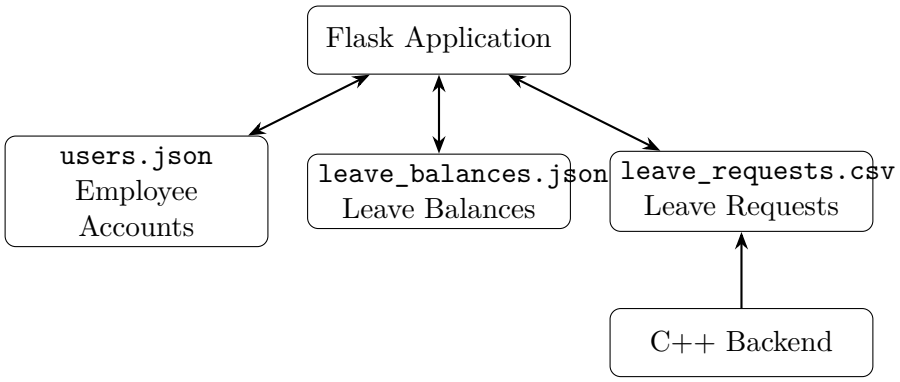



Figure 1.14: Data storage and interaction between the application components

## 1.5 Testing and Results

The LeaveFlow application was tested to verify its main features and the interaction between the frontend, Flask application, C++ backend, and local data files.

Table 1.1: Functional testing of LeaveFlow

| S.No. | Test Case | Expected Result |
|---|---|---|
| 1 | User login | The user accesses the appropriate interface after successful authentication. |
| 2 | Submit leave request | A valid request is stored with `PENDING` status. |
| 3 | Approve leave request | The status changes to `APPROVED`, and the available leave balance is reduced. |
| 4 | Reject leave request | The status changes to `REJECTED` without deducting leave. |
| 5 | View request history | The employee can view the status of their submitted requests. |
| 6 | Role-based access | Administrative operations are restricted to authorized administrators. |
| 7 | View leave balance | The employee can view updated casual, sick, and earned leave balances. |

The screenshots demonstrate the application's login interface, pending requests, approval confirmation, request history, and updated leave balances. In the demonstrated two-day sick-leave approval, the available sick-leave balance decreased from 10 days to 8 days.

The results show that the major application workflows operate across the web interface, Flask application, C++ backend, and local data files. The test cases cover successful actions, such as login and approval, as well as access and status checks, such as role restrictions and rejection without balance deduction. Together, these checks provide evidence that the main employee and administrator workflows are connected end to end.

## 1.6 Additional Features

### 1.6.1 Employee Account Creation

LeaveFlow allows administrators to create employee accounts through the employee management interface. The administrator enters the employee's name and a temporary password. The application creates the account and initializes the default casual, sick, and earned leave balances. This feature simplifies employee onboarding.

### 1.6.2 Leave Request Filtering

Administrators can filter leave requests by status, including All Requests, Pending, Approved, and Rejected. This makes it easier to find requests that need attention and review previously processed applications.

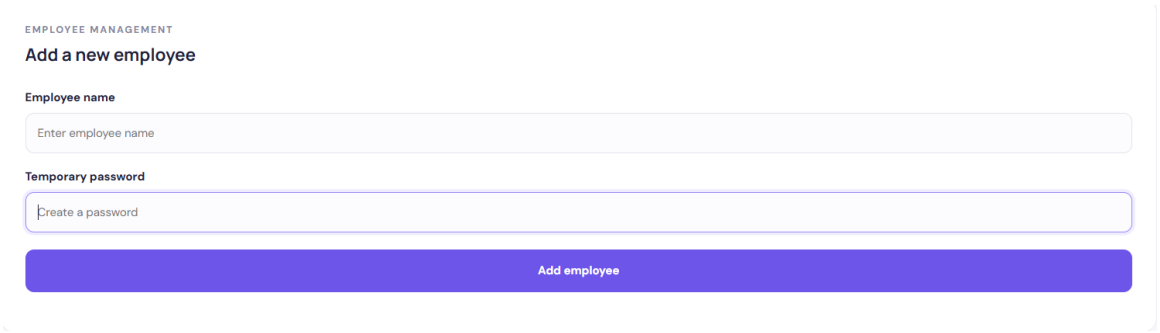



Figure 1.15: Interface for adding a new employee




Figure 1.16: Filtering leave requests by status

### 1.6.3 Administrative Summary Dashboard

The administrative dashboard summarizes leave activity, including the total number of requests and the numbers of approved and rejected applications. It also displays pending requests, helping the administrator quickly understand the current state of leave processing.

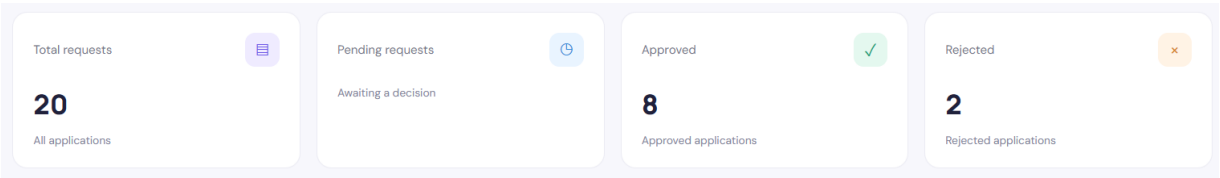



Figure 1.17: Administrative dashboard showing leave request statistics

### 1.6.4 C++ and Flask Integration

Flask invokes the compiled C++ executable with the employee ID, leave type, start date, end date, and reason. The executable processes the arguments and returns output and an exit status. Flask uses this result to decide whether to display a success or error message to the employee. This integration allows the web interface and C++ validation logic to work together.

```
result = subprocess.run(
    ["backend/leave_backend.exe",
     employee_id, leave_type,
     start_date, end_date, reason],
    capture_output=True,
    text=True
)

if result.returncode == 0:
```

```
    message = result.stdout
else:
    message = result.stderr or result.stdout
```

The snippet is illustrative and should be checked against the actual Flask route before being described as an exact extract from the source code.

### 1.6.5 Date and Leave-Duration Validation

LeaveFlow incorporates date validation and leave-duration calculation in its C++ backend. The abstract `Leave` class defines the common structure and operations for different leave types. Its `calculateDays()` function parses the start and end dates using `std::get_time()`, converts them into time values using `std::mktime()`, and calculates the difference between the dates.

The function returns zero if date parsing or time conversion fails. For valid conversions, it calculates the duration in days and adds one to include both the start and end dates.

```
int Leave::calculateDays() const {
    std::tm start = {};
    std::tm end = {};

    std::istringstream startInput(startDate);
    std::istringstream endInput(endDate);

    startInput >> std::get_time(&start, "%Y-%m-%d");
    endInput >> std::get_time(&end, "%Y-%m-%d");

    if (startInput.fail() || endInput.fail()) {
        return 0;
    }

    start.tm_isdst = -1;
    end.tm_isdst = -1;

    std::time_t startTime = std::mktime(&start);
    std::time_t endTime = std::mktime(&end);

    if (startTime == static_cast<std::time_t>(-1) ||
        endTime == static_cast<std::time_t>(-1)) {
        return 0;
    }

    double seconds = std::difftime(endTime, startTime);
    int days = static_cast<int>(
        seconds / (60 * 60 * 24)
    );

    return days + 1;
}
```

The maximum permitted duration is defined separately for each leave type through polymorphism. For example, the `CasualLeave` class restricts casual leave to three days.

```
int CasualLeave::getMaximumDays() const {
    return 3;
}
```

backend > src > Leave.cpp > ...

```cpp
#include "../include/Leave.h"

#include <ctime>
#include <iomanip>
#include <sstream>

Leave::Leave(int leaveId,
             const std::string& startDate,
             const std::string& endDate,
             const std::string& reason)
    : leaveId(leaveId),
      startDate(startDate),
      endDate(endDate),
      reason(reason) {
}

int Leave::calculateDays() const {
    std::tm start = {}; //std::tm creates containers for storing values in y-m-d form
    std::tm end = {};

    std::istringstream startInput(startDate); //istringstream takes input from already
    std::istringstream endInput(endDate);     //stored string instead of keyboard

    startInput >> std::get_time(&start, "%Y-%m-%d"); //get_time takes string values in
    endInput >> std::get_time(&end, "%Y-%m-%d");     //year month date format

    if (startInput.fail() || endInput.fail()) {
        return 0;
    }

    start.tm_isdst = -1;
    end.tm_isdst = -1;
```

Figure 1.18: Date parsing and validation in the calculateDays() function

```
32      end.tm_isdst = -1;
33
34      std::time_t startTime = std::mktime(&start); //Converts into time format for taking
35      std::time_t endTime = std::mktime(&end);     //difference
36
37      if (startTime == static_cast<std::time_t>(-1) || //Checks if time conversion fails
38          endTime == static_cast<std::time_t>(-1)) {
39          return 0;
40      }
41
42      double seconds = std::difftime(endTime, startTime);  //Gives difference in times.
43      int days = static_cast<int>(seconds / (60 * 60 * 24));
44
45      return days + 1;
46  }
47
48  int Leave::getLeaveId() const {
49      return leaveId;
50  }
51
52  std::string Leave::getStartDate() const {
53      return startDate;
54  }
55
56  std::string Leave::getEndDate() const {
57      return endDate;
58  }
59
60  std::string Leave::getReason() const {
61      return reason;
62  }
```

Figure 1.19: Time conversion and leave-duration calculation in Leave.cpp

backend > include > C Leave.h > Leave > ~Leave()

```
1
2   #ifndef LEAVE_H
3   #define LEAVE_H
4
5   #include <string>
6
7   class Leave {
8   protected:
9       int leaveId;
10      std::string startDate;
11      std::string endDate;
12      std::string reason;
13
14  public:
15      Leave(int leaveId,
16            const std::string& startDate,
17            const std::string& endDate,
18            const std::string& reason);
19
20      virtual int calculateDays() const;
21      virtual std::string getLeaveType() const = 0;
22      virtual void display() const = 0;
23      virtual int getMaximumDays() const = 0;
24
25      int getLeaveId() const;
26      std::string getStartDate() const;
27      std::string getEndDate() const;
28      std::string getReason() const;
29
30      virtual ~Leave() = default;
31  };
32
33  #endif
34
```

Figure 1.20: Abstract Leave class with virtual functions

backend > src > CasualLeave.cpp > display() const

```cpp
#include "../include/CasualLeave.h"
#include <iostream>

CasualLeave::CasualLeave(int leaveId,
                         const std::string& startDate,
                         const std::string& endDate,
                         const std::string& reason)
    : Leave(leaveId, startDate, endDate, reason) {
}

std::string CasualLeave::getLeaveType() const {
    return "Casual Leave";
}

int CasualLeave::getMaximumDays() const {
    return 3;
}

void CasualLeave::display() const {
    std::cout << "Leave ID: " << leaveId << '\n';
    std::cout << "Leave Type: " << getLeaveType() << '\n';
    std::cout << "Start Date: " << startDate << '\n';
    std::cout << "End Date: " << endDate << '\n';
    std::cout << "Reason: " << reason << '\n';
}
```

Figure 1.21: CasualLeave class defining the maximum permitted duration

This design keeps the common date-calculation logic in the base class while allowing derived classes to specify their own leave-duration limits. The same approach can be used for sick leave and earned leave.

## 1.7 Conclusion

LeaveFlow is a web-based Employee Leave Management System developed to simplify and streamline the leave application and approval process. It provides separate interfaces for employees and administrators, enabling employees to apply for leave, view their leave balances, and track request statuses, while administrators can manage employee accounts and process leave requests.

The system integrates HTML and CSS for the user interface, Flask for application logic, and C++ for leave validation and duration calculation. JSON and CSV files are used for data storage. The project demonstrates object-oriented programming concepts such as abstraction, inheritance, encapsulation, and polymorphism, along with integration between different programming technologies.

The successful implementation of the major workflows demonstrates how LeaveFlow can make leave management more organized, efficient, and transparent.